

技术支持工程师 · 深度题库

广州云趣信息科技有限公司
(全新 20 道 · 与上一版完全不重复)

☑ 本版亮点：

- 避开上一版所有 23 道题，深挖 JD 每条任职要求的第二层考点
- 包含：MySQL redo/undo 两阶段提交、MVCC Read View、达梦数据泵、ProxySQL 实战配置、Compose 完整 YAML、K8s 三种探针、STP 安全特性、OSPF DR/BDR/NSSA、SYN 洪水防御、H3C NAT+OSPF 联动、MGR 切换后业务故障排查、客户强推全量发布的沟通策略、TIME_WAIT 根治
- 每题配：☑ 人话解释 + ☑ 面试模板 + ☑☑ 速记口诀

二、数据库硬技能（5 道）

☑☑ 本方向共 5 道深度题，全部是上一版没出的点。

☑ [2-深度-1] 二.1 MySQL 内核原理

上次只提了 redo/undo 的名字，这次深入——面试官如果问你「InnoDB 崩溃恢复到底怎么实现的」就考这个。

☑ MySQL 的 redo log、undo log、binlog

三者分别存什么？提交事务时的「两阶段提交」又是啥？

☑ 人话解释（看完必懂）：想象你在银行转账（提交事务）：

- undo log = 转账前先拍一张「原始状态照片」，万一转账失败能恢复回去（回滚）；
- redo log =

转账时把每一步操作都记在「便签条」上，还没来得及记到总账本（数据文件）就死机了，重启后根据便签条把转账补完（重放，崩溃恢复）；

- binlog = 转账结束后写进「正式流水记录」，给主从复制用（从库靠这个同步）；
- 两阶段提交：

① redo log prepare（转账动作写进便签条，说「我准备提交了」）；

② binlog commit（写正式流水）；

③ redo log commit（便签条标记「已完成」）。

为啥要两阶段？如果先写 binlog 后写

redo，中间挂了——从库已经收到流水了，但主库自己没做完，数据就不一致了。

两阶段保证 redo 和 binlog 要么都有要么都没有。

☒ 面试标准作答：三者定位：

- redo log (InnoDB 专属)：物理日志，记录「某个数据页做了什么修改」（比如页号 + 偏移 + 新值），循环写固定大小 (innodb_log_file_size)。崩溃恢复时重放 redo 保证已提交事务不丢。

- undo log (InnoDB 专属)：逻辑日志，记录「修改前的旧值」。两个用途：① 事务回滚 (ROLLBACK)；② MVCC 多版本并发控制 (其它事务读旧值)。undo 会被 purge 线程清理。

- binlog (Server 层，所有引擎通用)：逻辑日志，记录「SQL 语句 / 行变更」。用于主从复制、数据恢复 (mysqlbinlog

回放时间点)。三种格式：statement(记SQL)/row(记行变更)/mixed(混合)，生产用 row。

两阶段提交 (2PC)：提交事务时 redo 写 prepare → binlog 写 → redo 写 commit。保证 redo 和 binlog 的一致性——崩溃恢复时如果 redo 是 prepare 状态但 binlog 有，就 commit；如果 redo prepare 但 binlog 没有，就回滚。这是 MySQL 保证主从不丢数据的关键。

☒☒ 速记口诀：三日志记忆法：undo=回滚，redo=重做，binlog=复制；两阶段保证 redo/binlog 一致。

☒ [2-深度-2] 二.1 MySQL 内核原理

MVCC 是 InnoDB 的灵魂，上次只提了名字，这次是面试官最爱追着问的。

☒ InnoDB 的 MVCC 多版本并发控制到底怎么实现的？Read View 是什么？RC 和 RR 隔离级别的区别就是 Read View 的生成时机？

☒ 人话解释 (看完必懂)：把 MVCC 想成老照片+时间戳：

- 每行数据都有两个隐藏字段：创建版本号 trx_id (哪个事务改的) + 删除版本号 (哪个事务删的，初始 NULL)；

- 每次事务开始会拿到自己的「时间戳」（其实是自增的事务 ID）；

- 其它事务查这行的时候，发现创建版本号比自己新——说明我开始之后有人改过了，我看不到，就顺着 undo log 往前找「上一张照片」（旧版本）；

- Read

View (读视图) 就是事务开始瞬间拍的一张「全局快照」——记录了「当时所有活跃的事务 ID」，读的时候对照：如果这行的修改者在我的快照里是活跃的，我就不认它，找旧版本。

RC vs RR 的区别：

- RC (读提交)：每次 SELECT 都生成新的 Read View——所以别人刚 commit 的修改，我下一条 SELECT 就能看到 (不可重复读)；

- RR (可重复读)：事务开始时只生成一次 Read

View——所以整个事务期间不管别人怎么改，我看到的都是同一份快照 (MySQL 默认级别，解决了不可重复读)。

☒ 面试标准作答：MVCC 实现要素：

① 每行隐藏字段：DB_TRX_ID (最近修改事务 ID)、DB_ROLL_PTR (指向 undo 链表)；

② undo log 链：同一行的不同版本通过 roll_ptr 串成链表，形成版本链；

③ Read View：事务查询时生成的快照，包含

m_ids (活跃事务列表)、min_trx_id、max_trx_id、creator_trx_id；

④ 可见性规则：某版本对当前事务可见 ⇔ 其 trx_id 不在 Read View

的活跃列表中（即修改者在我开事务前就已提交）。

RC vs RR:

- RC: 每条 SELECT 创建新 Read View → 不可重复读（同一事务两次 SELECT 结果不同）；
- RR: 事务第一条 SELECT 创建 Read View 后复用 → 可重复读。

注意: MVCC 只解决读写冲突, 写写冲突（两个事务同时改一行）还是锁解决。

☑☑ 速记口诀: MVCC 三要素: 隐藏字段 + undo 链 + Read View。RC 每次读都建新快照; RR 建一次用到底。

☑ [2-深度-3] 二.1 PostgreSQL

上次只对比了功能, 这次挖底层。面试官能看出你是不是真懂 PG。

☑ PostgreSQL 的 WAL 日志是什么? 和 MySQL redo 有啥本质区别? PG 的表空间 (Tablespace) 又是干嘛的?

☑ 人话解释 (看完必懂): • WAL (Write-Ahead Logging) 就是 PG 版的 redo log, 但设计思路完全不一样:

MySQL redo 记的是「某个页的偏移改了多少字节」(物理日志);

PG WAL 记的是「哪个事务 ID 做了什么操作」(其实是介于物理和逻辑之间, 比如 XLogInsert 插入一条 tuple 记录)。

关键差异: PG 的数据文件本身不存 undo (PG 的 MVCC 是每个 tuple 自带 xmin/xmax 版本字段, 旧版本直接留在数据页里, 靠 VACUUM 清理), 所以 PG 不需要 redo 之外额外有 undo log。

- 表空间 Tablespace: PG 允许你把不同的表/索引放到不同的磁盘路径上。

比如大表放 NVMe 盘、冷表放 SATA 盘, MySQL 没有这个功能 (MySQL 是按库分目录在同一个数据目录下)。

创建: CREATE TABLESPACE fast DISK '/data/fast_disk'; 然后 CREATE TABLE big_tbl (...) TABLESPACE fast;

☑ 面试标准作答: WAL vs redo:

- MySQL InnoDB

redo: 纯物理日志, 按页号+偏移+新值记录, 固定大小循环写 (innodb_log_file_size); undo 独立存储。

• PostgreSQL WAL: 介于物理与逻辑之间, 每个操作生成 XLOG 记录 (TransactionID + 操作类型 + 数据), 没有独立的 undo log (旧版本 tuple 直接留在数据页中, 通过 xmin/xmax 字段标记可见性, 靠 VACUUM 清理)。WAL 采用无限追加 (可配置 wal_keep_segments), 支持流式备份 (pg_basebackup + WAL 归档 = PITR 时间点恢复)。

表空间: PG 的 Tablespace 允许指定物理存储路径 (不同磁盘/不同 RAID

组), 实现冷热分层存储 (热表放 NVMe、冷表放 SATA)。MySQL 通过 innodb_file_per_table + 符号链接能部分实现, 但不原生。

☑☑ 速记口诀: PG 没有 undo log! MVCC 靠 tuple 的 xmin/xmax + VACUUM 清理。表空间是 PG 独有功能。

☑ [2-深度-4] 二.1 + 二.3 MGR + ProxySQL

上次只说了 ProxySQL 的定位，这次要实战——JD 要的是「熟练使用」。

☒ 写一段 ProxySQL 的配置：1 主 2 从的读写分离，分片键 user_id 取模分成 4 片，并配健康检查。

☒ 人话解释（看完必懂）：ProxySQL 的配置存 SQLite，用 admin 接口改（默认 6032 端口）。核心概念：

- mysql_servers：后面的 MySQL 节点列表（hostgroup 10=写组，hostgroup 20=读组）；
- mysql_replication_hostgroups：指定主从复制的主/从组，ProxySQL 自动探测主从切换 hostgroup；
- mysql_query_rules：SQL 路由规则，按正则匹配后转发到对应 hostgroup；
- mysql_server_group / mysql_server_group_members：定义分片组 + 成员。

操作步骤全是 SQL（因为 ProxySQL 管理端就是个 SQLite）：

```
```sql
```

```
-- 1. 加后端节点（hostgroup 10=写，20=读）
```

```
INSERT INTO mysql_servers(hostgroup_id,hostname,port,weight)
```

```
VALUES (10,'10.0.1.11',3306,100),
```

```
(20,'10.0.1.12',3306,100),
```

```
(20,'10.0.1.13',3306,100);
```

```
-- 2. 声明主从组关系（ProxySQL 自动探测写哪个 hostgroup）
```

```
INSERT INTO mysql_replication_hostgroups(values,writer_hostgroup,backup_writer_hostgroup,
reader_hostgroup,offline_hostgroup)
```

```
VALUES ('read_only',10,30,20,40);
```

```
-- 3. 读规则：SELECT 走读组
```

```
INSERT INTO mysql_query_rules(rule_id,active,match_digest,destination_hostgroup,apply)
```

```
VALUES (1,1,'^SELECT.*',20,1);
```

```
-- 4. 写规则：INSERT/UPDATE/DELETE 走写组（0=默认写组10）
```

```
INSERT INTO mysql_query_rules(rule_id,active,match_digest,destination_hostgroup,apply)
```

```
VALUES (2,1,'^(INSERT|UPDATE|DELETE).*',10,1);
```

```
LOAD MYSQL VARIABLES TO RUNTIME; SAVE MYSQL VARIABLES TO DISK;
```

```
LOAD MYSQL SERVERS TO RUNTIME; SAVE MYSQL SERVERS TO DISK;
```

```
LOAD MYSQL QUERY RULES TO RUNTIME; SAVE MYSQL QUERY RULES TO DISK;
```

```
```
```

☒ 面试标准作答：ProxySQL 配置通过 admin 接口（6032 端口）操作 SQLite。完整 1 主 2 从读写分离 + 分片：

① mysql_servers：注册后端节点，hostgroup 10=写组，20=读组；

② mysql_replication_hostgroups：声明主从组（参数 values='read_only' 表示 ProxySQL 会监控 show variables like 'read_only'，read_only=OFF 是主，ON 是从，自动维护 hostgroup 归属）；

③ mysql_server_group / mysql_server_group_members：定义分片组（group 101~104 分别对应 4 个分片，hostgroup 101 = shard1 的主+从组合）；

④ mysql_query_rules：用 match_digest 正则匹配 SQL，destination_hostgroup

指向目标组。分库分表的路由逻辑可以在应用层做（推荐 ShardingSphere-JDBC），也可以用 ProxySQL 的 multiplex 模式 + mysql_sharding_group_members_table 配合。

健康检查：mysql_monitor_connect_interval / mysql_monitor_ping_interval / mysql_monitor_read_only_interval，ProxySQL 自动探测节点存活和主从状态变化。

☒☒ 速记口诀：ProxySQL 配置=SQL！记住 hostgroup 的概念（10=写，20=读）+ mysql_replication_hostgroups 自动探测主从。

☒ [2-深度-5] 二.2 达梦数据库

上次只说了达梦的备份工具 dmrman，这次挖迁移工具链——JD 明确提了「熟悉迁移工具链」。

☒ 达梦数据库的 dexp/dimp 数据泵和 Oracle 的 expdp/impdp 是一回事吗？达梦的 DMETL 工具做什么的？

☒ 人话解释（看完必懂）：达梦的工具链直接抄 Oracle 命名，原理类似但有差异：

- dexp/dimp = Oracle 的 expdp/impdp（Data Pump 数据泵），比老的 exp/imp 快 10 倍，因为是直接读写数据文件不用走 SQL

层。支持按表、按用户、按整个库导出导入，支持并行。

dexp 命令示例：`dexp USERID=SYSDBA/SYSDBA FILE=/tmp/test.dmp LOG=/tmp/test.log OWNER=TEST DIRECTORY=BACKUP\_DIR COMPRESS=Y`

注意要先在达梦里 `CREATE OR REPLACE DIRECTORY BACKUP\_DIR AS '/tmp';`（Oracle 也是这样）。

- DMETL = 达梦的 ETL 工具（Data Migration / Extract Transform Load），比单纯 dexp 更强大——可以做 Oracle/MySQL → 达梦

的跨库迁移，支持数据清洗、类型映射、批量同步。信创项目里常用 DMETL 把老 Oracle 系统迁到达梦。

☒ 面试标准作答：达梦迁移工具链：

① dexp/dimp：逻辑备份/恢复工具，对标 Oracle

expdp/impdp。支持全库/按用户/按表导出，并行度 PARALLEL=n，支持 COMPRESS 压缩。DIRECTORY 对象必须先数据库侧创建（CREATE OR REPLACE DIRECTORY）。

② dmrman：物理备份/恢复（对标 Oracle

RMAN），支持增量备份、表空间级备份、归档模式控制。

③ DTS（达梦迁移工具）：图形化跨库迁移，支持 Oracle/MySQL/SQL Server → 达梦

的结构+数据+索引+约束+触发器迁移，自动做类型映射（比如 NUMBER(10) → INT，VARCHAR2 → VARCHAR）。

④ DMETL：企业级 ETL，做实时/批量数据同步、跨库数据清洗。

迁移实战：先 DTS 迁结构 → DMETL 迁数据 → dexp 校验导入导出一致性。达梦语法兼容 Oracle 大部分场景，迁移成本不高。

☒☒ 速记口诀：达梦工具抄 Oracle：dexp=expdp，dmrman=RMAN，DTS=跨库迁，DMETL=ETL。

三、云原生与虚拟化（4 道）

☒☒ 本方向共 4 道深度题，全部是上一版没出的点。

☒ [3-深度-1] 三.1 Docker

上次只说了镜像和 Compose，这次挖网络——面试官能看出你是不是只会 docker run。

☒ Docker 的四种网络模式（bridge/host/none/container）分别啥意思？容器间如何实现自定义网络互通？

☒ 人话解释（看完必懂）：把四种网络模式想成四种入住方式：

- bridge（默认）：住独立小隔间——每个容器有自己的 IP（172.17.0.x），通过 docker0 虚拟网桥和宿主机通信，容器间能互相 ping（只要在同一个 network）。
- host：住房东自己房间——直接用宿主机的网络栈，容器 IP=宿主机 IP，端口冲突（容器开 80 端口会占宿主机 80），性能最好。
- none：住没网的小黑屋——完全没有网络，loopback 都没有，适合做安全隔离。
- container：住朋友房间的沙发——共享另一个容器的网络栈，两个容器 IP 完全一样（典型：sidecar 模式，比如 Istio proxy 和业务容器共享网络）。

- 自定义网络互通：``docker network create mynet`` → ``docker run --name web --network mynet ...`` → ``docker run --name db --network mynet ...`` → 容器间直接用容器名互 ping（DNS 解析），比 bridge 默认网络好（默认 bridge 不支持容器名 DNS）。

☒ 面试标准作答：四种模式对比：

| 模式 | 说明 | 场景 |

|-----|-----|-----|

| bridge | 默认，docker0 桥接，独立容器 IP | 大多数单容器场景 |

| host | 共享宿主机网络，无网络隔离 | 高性能需求（网络插件/监控探针） |

| none | 无网络 | 安全隔离容器 |

| container:xxx | 共享某容器的网络栈 | Sidecar（Istio envoy 共享业务容器网络） |

自定义网络：bridge 模式的问题是默认不支持容器名 DNS 解析 + 容器重启 IP 会变。用 ``docker network create -d bridge mynet`` 创建自定义 bridge 网络，加入同一网络的容器可以用容器名互 ping（内置 DNS）。多容器编排（Compose/K8s）都是自动创建自定义网络。

跨宿主机容器网络：需要 Overlay 模式（Swarm/K8s Calico/Flannel），通过 VXLAN 隧道跨节点通信。

☒☒ 速记口诀：bridge=独立隔间；host=共享房间；none=没网；container=共享朋友家。自定义 bridge 支持容器名 DNS。

☒ [3-深度-2] 三.1 Docker Compose

上次只说了 Compose 定位，这次要动手写——JD 要「多容器编排」。

☒ 写一个完整的 docker-compose.yml：web（Flask）+ db（MySQL）+ redis 三服务，带 volumes 持久化、depends_on 依赖、healthcheck，还要配 MySQL 只读副本？

☒ 人话解释（看完必懂）：直接给你一个能跑的示例，每行带人话解释：

```
```yaml
```

```
version: '3.8'
```

```
services:
```

```
web:
```

```
build: ./web # web 目录下有 Dockerfile，自己构建镜像
```

```
ports:
```

```
- "5000:5000" # 宿主机 5000 → 容器 5000
```

```
environment: # 容器环境变量
```

```
- MYSQL_HOST=db
```

```
- REDIS_HOST=redis
```

```
depends_on: # 启动顺序
```

```
db:
```

```
condition: service_healthy # 等 db 健康检查过了才启动
```

```
redis:
```

```
condition: service_started
```

```
restart: always
```

```
db:
```

```
image: mysql:8.0
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD: root123
```

```
MYSQL_DATABASE: mydb
```

```
volumes:
```

```
- mysql_data:/var/lib/mysql # 数据持久化
```

```
- ./init.sql:/docker-entrypoint-initdb.d/init.sql # 首次启动时执行
```

```
command: --server-id=1 --log-bin --binlog-format=ROW
```

```
healthcheck:
```

```
test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
```

```
interval: 10s
```

```
timeout: 5s
```

```
retries: 5
```

```
db_slave: # 只读副本
```

```
image: mysql:8.0
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD: root123
```

```
volumes:
```

```
- mysql_slave_data:/var/lib/mysql
```

```
command: > # 多行命令
--server-id=2
--read-only=1
--log-bin
--binlog-format=ROW
depends_on:
db:
condition: service_healthy
```

```
redis:
image: redis:7-alpine
volumes:
- redis_data:/data
command: redis-server --appendonly yes # AOF 持久化
healthcheck:
test: ["CMD", "redis-cli", "ping"]
```

```
volumes: # 声明命名卷
mysql_data:
mysql_slave_data:
redis_data:
``,``,``
```

启动：docker-compose up -d（后台跑）。mysql\_slave 的复制还要手动在 MySQL 里执行 CHANGE MASTER TO 配一下 binlog 位点。

☒ 面试标准作答：完整配置如上。

关键概念：

- volumes 两种写法：命名卷 mysql\_data → Docker 管，数据在 /var/lib/docker/volumes/；绑定挂载 ./init.sql → 宿主机路径直接挂进容器；
- depends\_on + condition：condition: service\_healthy 比单纯 service\_started 强——等 db 的 healthcheck 真正 OK 了 web 才启动，避免 MySQL 还没 ready；
- command：覆盖镜像默认的 CMD，MySQL 主从模式用不同的 --server-id 和 --read-only；
- healthcheck：让 Compose 知道服务是否真的准备好了，配合 depends\_on 使用；
- db\_slave 复制配置：docker-compose up 后需要手动执行一次——mysql -h db -uroot -proot123 -e "SHOW MASTER STATUS;" 拿到 File+Position → mysql -h db\_slave ... 执行 CHANGE MASTER TO ... START SLAVE。

生产中建议用 docker-entrypoint-initdb.d 挂载初始化脚本，让首次启动自动配好复制。

☒☒ 速记口诀：四关键字：volumes 持久化、depends\_on 顺序、healthcheck 就绪检查、command 覆盖参数。db 主从用 server-id + read-only。

### ☒ [3-深度-3] 三.2 Kubernetes

上次只说了 CrashLoopBackOff 排查，这次深挖探针——JD 要熟悉 K8s 技术栈，探针是核心。

☒ K8s 的三种探针（liveness/readiness/startup probe）到底啥区别？我的 Pod 一直在 CrashLoopBackOff，但 liveness probe 是好的——这是咋回事？

☒ 人话解释（看完必懂）：把三种探针想成三种不同的考试：

- startupProbe（初试）：测「这个学生能不能醒过来参加考试？」——应用启动慢（比如 Java 微服务冷启动 60 秒）时用，在 startupProbe 成功之前不会跑另外两个探针，给应用足够时间启动。

- livenessProbe（健康体检）：测「这个学生是不是还活着？」——如果挂了 K8s 就杀进程重启 Pod（RestartPolicy=Always）。用来抓死锁/无限循环这种「活着但废了」的情况。

- readinessProbe（预备铃）：测「这个学生能不能开始答卷了？」——没通过的话 K8s 把 Pod 从 Service 的 endpoints 里摘掉（流量不发过来了），但不杀 Pod。用来做优雅下线（应用要重启了先标记 notReady 让流量平滑切走）。

- 场景题：Pod 一直在 CrashLoopBackOff 但 liveness 是好的——说明容器启动后马上退出了（CMD 执行完了），liveness 探针都还没来得及跑，容器就已经挂了。典型原因：Dockerfile 的 CMD 写错（比如写成 echo 而不是 python app.py）、进程启动后立即退出。

排查：`kubectl describe pod` 看 Events → `kubectl logs --previous` 看上一次容器崩溃日志。

☒ 面试标准作答：三种探针定位：

| 探针 | 没通过的动作 | 解决什么问题 |

|-----|-----|-----|

| startupProbe | 等待（超时杀 Pod） | 慢启动应用的「宽限期」，避免 liveness 过早杀 |

| livenessProbe | 杀 Pod 重启 | 死锁/僵死/无限循环（进程活着但功能失效） |

| readinessProbe | Service 摘 endpoints | 优雅下线/滚动更新时的流量平滑 |

配置示例：livenessProbe 用 httpGet（HTTP 探针）或 exec（执行命令），initialDelaySeconds=应用启动时间，periodSeconds=检测间隔，failureThreshold=失败 N 次触发动作。

CrashLoopBackOff + liveness OK 的根因：容器主进程（PID 1）启动后立即退出，K8s

还没来得及跑 liveness 探针容器就死了，所以 liveness 显示

OK（其实是还没开始检测）。典型：Dockerfile CMD 写成了 shell

命令的执行结果（echo）而不是前台进程；或者应用启动抛异常直接退出。用 `kubectl logs -c --previous` 看上一次崩溃日志是关键。

☒☒ 速记口诀：startup=等它醒；liveness=死了重启；readiness=准备好接流量。CrashLoopBackOff 说明主进程秒退，看 --previous 日志。

### ☒ [3-深度-4] 三.2 Kubernetes

上次说了 K8s 组件，但没讲 Service 四种类型的场景——这是 K8s 最容易被问到的。

☒ K8s 的四种 Service

类型（ClusterIP/NodePort/LoadBalancer/ExternalName）分别什么时候用？Ingress

又是什么关系？

☒ 人话解释（看完必懂）：把四种 Service 想成四种接客前台：

- ClusterIP（默认）：只接待内部员工——只有 K8s 集群内部的 Pod 能访问，外部人（互联网/公司网络）访问不到。典型用途：后端服务之间互相调用。
- NodePort：在每个小区门口贴个告示（每台 K8s 节点机开一个 30000-32767 的端口），外部人敲任意节点 IP+30080 就能进来。生产一般不用（端口号难记、不能做 TLS 终止）。
- LoadBalancer：在大门口装一个呼叫中心——云厂商会自动帮你创建一个公网 IP+负载均衡器，流量先经过云 LB 再进 K8s。AWS ELB / Azure ALB / GCP LB 都支持。
- ExternalName：直接指给你另一家酒店的电话——相当于 CNAME，把请求转到集群外的某个域名。

• Ingress：ClusterIP 是「内部前台」，Ingress 是「外部总前台」——先让外部流量到 Ingress Controller（nginx-ingress / traefik），Ingress 根据 URL/域名/path 路由到不同的 Service，还能配 TLS 证书、限流、跨域。生产环境基本都是 Ingress + ClusterIP 组合。

☒ 面试标准作答：Service 类型：

| 类型 | 访问范围 | 典型场景 |

|-----|-----|-----|

| ClusterIP | 集群内 | 服务间内部调用（默认） |

| NodePort | 外部可访问，端口 30000-32767 | 临时调试、内网穿透 |

| LoadBalancer | 云厂商自动分配公网 IP | 云环境对外暴露单 Service |

| ExternalName | 转到外部 DNS | 访问外部服务（legacy 迁移） |

Ingress = 七层路由入口：在 Service 之前加一层，根据 Host 头和 URL

路径匹配规则路由。优势：① 一个公网 IP + 多个域名/路径 → 多个 Service；② TLS

证书集中管理；③ 限流/跨域/重写规则。Ingress Controller 是 DaemonSet 或

Deployment（nginx-ingress / Traefik / Istio IngressGateway）。生产架构：公网流量 →

Ingress（TLS 终止）→ Service ClusterIP → Pod。

☒☒ 速记口诀：ClusterIP=内部；NodePort=临时；LoadBalancer=云厂商；ExternalName=CNAME。Ingress=七层路由+TLS+域名。

## 四、监控与自动化（3 道）

☒☒ 本方向共 3 道深度题，全部是上一版没出的点。

### ☒ [4-深度-1] 四.1 Zabbix

上次只说了 Zabbix 是 Agent 推送，这次要实战——JD 要「精通」。

☒ Zabbix 的 Item/Trigger/Action/Graph 四层对象怎么关联起来工作的？写一个监控 MySQL 主从延迟超过 10 秒就告警的完整 Trigger 表达式。

☒ 人话解释（看完必懂）：把 Zabbix 四层想成四步流水线：

- ① Item（采集项）：「我要收什么数据？」——比如 mysql.slave\_lag  
是从库复制延迟，采集键就是 `mysql.slave.lag[Slave\_IO\_Running,Slave\_SQL\_Running]`；
- ② Trigger（触发器）：「收到的数据满足什么条件算异常？」——比如  
`last(/MySQL-Master/mysql.slave.lag)>10`；
- ③ Action（动作）：「异常了之后怎么办？」——发邮件 / 发钉钉 Webhook / 执行脚本；
- ④ Graph（图表）：「数据怎么可视化？」——在 Monitoring → Graphs 里配置。

完整 Trigger 表达式（Zabbix 6.0 新版）：

```

```
{MySQL-Master:mysql.slave.lag.last(0)}>10  
AND {MySQL-Master:mysql.slave.lag.last(0)}<>0
```

```

解释：MySQL-Master 是 Host 名字，mysql.slave.lag 是 Item key，last(0) 是最后一个值。加 `<>0` 是为了排除主从完全断开时 slave\_lag=0（还没同步过）的误报。

☒ 面试标准作答：四层关联：Item 定义采集指标 → Trigger 引用 Item 值写阈值表达式 → Trigger 触发 PROBLEM → Action 匹配 EventSeverity+Trigger 调用 发送媒介（Media=邮件/钉钉/企微） → 通知 User Group。

MySQL 主从延迟 Trigger（Zabbix 5.x 语法）：

```

```
{Template MySQL Slave:mysql.slave.lag.last()} > 10
```

```

或 Zabbix 6.0 新语法（支持函数链）：

```

```
last(/MySQL-Template/mysql.slave.lag) > 10
```

```

注意事项：

- 模板里的 Item key 要和 mysql-agent 或 zabbix-agent 的 UserParameter 对上：UserParameter=mysql.slave.lag,mysql-NBe "SHOW SLAVE STATUS\G" | grep Seconds\_Behind\_Master | awk '{print \$2}'
- 建议加 AND 条件排除 null 或 0 的场景（主从刚断开时 lag 可能是 null）；
- 加恢复条件：`nodata(/MySQL-Template/mysql.slave.lag,5m)` 5 分钟没数据也要告警。

☒☒ 速记口诀：Item=采数据，Trigger=设阈值，Action=告警，Graph=画图。Trigger 引用 Item 用 {主机:键.函数()}。

#### ☒ [4-深度-2] 四.1 Prometheus

上次只说了 Prometheus 有 Alertmanager，这次要写实际 YAML——JD 要熟练用。

☒ 写一个 Prometheus 的 Alerting Rules YAML，要求：MySQL 重启（up=0）5 分钟后发严重告警，CPU 超过 90% 持续 10 分钟发警告。还要带 recording rule（预计算热点指标）。

☒ 人话解释（看完必懂）：完整 Prometheus alertmanager\_rules.yml（注释版）：

```
` ``yaml
groups:
- name: mysql_alerts
 interval: 30s # 规则评估频率
 rules:
 # --- Recording Rule（预计算热点指标） ---
 - record: instance:node_cpu_usage:rate5m
 expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100
 annotations:
 summary: "预计算实例 5 分钟 CPU 利用率"

 # --- Alerting Rule ---
 - alert: MySQLDown
 expr: mysql_up == 0
 for: 5m # 持续 5 分钟才触发（防抖动）
 labels:
 severity: critical
 annotations:
 summary: "MySQL {{ $labels.instance }} 挂了 5 分钟"
 description: "mysql_up=0 持续 5m，业务不可用！"

 - alert: HighCPUUsage
 expr: instance:node_cpu_usage:rate5m > 90
 for: 10m
 labels:
 severity: warning
 annotations:
 summary: "实例 {{ $labels.instance }} CPU 超过 90% 持续 10 分钟"
 runbook: "https://wiki.com/cpu-hotspot"
 ` ``
```

生效：prometheus.yml → rule\_files → 重新加载 `curl -X POST localhost:9090/-/reload`。

Alertmanager 路由到钉钉/企微。

☒ 面试标准作答：PromQL 要点：

- rate(metric[5m])：计算 5 分钟窗口的每秒增长速率（counter 类型必用 rate）；
- avg by(instance)：按 instance 标签聚合；
- for: 5m：持续时间，防止瞬时抖动触发告警；
- Recording Rule 用 record 关键字，把常用表达式预计算成新指标，减少实时计算开销；

实战优化：

- 高基数标签（比如 url=每个请求路径）不要放 label，会把 TSDB 撑爆；
- 告警分级：critical(短信/电话) > warning(企微/钉钉) > info(邮件)；
- 配 inhibit：critical 告警出来时自动抑制同组的 warning 告警；
- Prometheus 内存不够 → 加 --storage.tsdb.retention.time 或接入 VictoriaMetrics/Thanos。

☒☒ 速记口诀：三关键字：record=预计算，alert=告警，for=持续时间防抖动。counter 类型必用 rate() 函数。

☒ [4-深度-3] 四.2 Shell 脚本

上次只出了备份脚本，这次是运维日常最常见的日志分析——JD 要 Shell 功底扎实。

☒ Shell 高级实战：用 awk 分析 nginx access.log，找出访问量 Top 10 的 IP + 每个 IP 的请求总数 + 平均响应时间。日志格式：\$remote\_addr - \$remote\_user [\$time\_local] "\$request" \$status \$body\_bytes\_sent "\$http\_referer" "\$http\_user\_agent" \$request\_time

☒ 人话解释（看完必懂）：nginx access.log 默认是空格分隔，IP 是第一列，\$request\_time 是最后一列。直接用 awk 一把梭：

```
```bash
awk '{
ip=$1; # 第一列=IP
req_time=$NF; # 最后一列=响应时间
cnt[ip]++; # 累加请求数
sum[ip]+=req_time; # 累加响应时间
}
END {
for (ip in cnt) printf "%-16s %6d %.3fs avg\n", ip, cnt[ip], sum[ip]/cnt[ip];
}' access.log | sort -k2 -rn | head -10
```
```

逐行人话解释：

- awk 默认按空格把每行拆成 \$1 \$2 \$3... 变量；
- cnt[ip] 和 sum[ip] 是 awk 的数组，用 IP 当 key；
- \$NF 表示最后一列（NF=Number of Fields）；
- END 块：全部行读完后执行一次；

- `sort -k2 -rn`: 按第 2 列（请求数）逆序排序，`head -10` 取前 10。

进阶版（排除静态资源 + 按状态码分组）：

```
```bash
awk '$7 !~ /\. (jpg|png|css|js)$ / && $9 ~ /^(4|5)/ {
cnt[$1,$9]++
} END { for (k in cnt) print k, cnt[k] }' access.log | sort -k3 -rn | head -20
```
```

意思：只看非静态资源 + 4xx/5xx 错误请求，找出错误最多的 IP+状态码组合。

☒ 面试标准作答：完整 awk 脚本如上。

awk 核心知识点：

- 内置变量：`$NF`（最后一列）、`NR`（当前行号）、`NF`（当前行列数）、`FILENAME`；
- 数组用法：`cnt[key]++`（累加计数器，key 可以是字段）；
- `BEGIN/END` 块：分别在文件读前/读完执行一次；
- 条件过滤：`$7 ~ /pattern/`（正则匹配）、`$7 !~`（反向匹配）、`$9 == 200`（精确比较）。

常见运维场景：

- 日志 Top N：`awk '{cnt[$1]++} END{for(k in cnt) print cnt[k],k}' | sort -rn | head`
- 错误率：`awk '$9 >= 500 {err++} END {print err/NR*100 "%"}'`
- 性能分析：`awk '{sum+=$NF; n++} END {print "avg=" sum/n}'`

☒☒ 速记口诀：awk 三板斧：数组 `cnt[key]++`、`$NF` 取最后一列、`sort + head` 排序取 TopN。

## 五、网络实操（5 道）

☒☒ 本方向共 5 道深度题，全部是上一版没出的点。

☒ [5-深度-1] 五.1 TCP/IP

上次只说了三次握手的流程，这次深入 seq 的作用 + SYN 洪水攻击——面试官最爱问的安全问题。

☒ TCP 三次握手里为什么客户端和服务端都要发 seq 序号？SYN 洪水攻击是什么原理？Linux 怎么防？

☒ 人话解释（看完必懂）：• seq 序号的本质：TCP

是面向字节流的协议——它把所有数据当成一串字节发送，

用 seq 给每个字节编号。比如我发了 100 字节给你，seq=1000 就是说从第 1000 号字节开始的 100 个字节。

为什么握手时就要发？因为双方需要在连接建立前就「对齐自己的初始序号」（ISN），不然后面收发数据时没法判断丢没丢、能不能重排。

• SYN 洪水攻击原理：我疯狂给你发 SYN=1 的包（伪造不同的源 IP），你的服务器收到 SYN 之后会分配一块内存（半连接队列 SYN queue）存这个「等对方回 ACK」的状态，但我永远不回 ACK，你的内存就被一堆假连接占满了，真的用户连不上来。

• Linux 怎么防：

```
```bash
```

```
sysctl -w net.ipv4.tcp_syncookies=1 # 开 syncookies（最关键！）
```

```
sysctl -w net.ipv4.tcp_max_syn_backlog=4096 # 加大半连接队列
```

```
sysctl -w net.ipv4.tcp_syn_retries=2 # SYN 重传次数少一点
```

```
sysctl -w net.ipv4.conf.all.rp_filter=1 # 反向路径过滤，丢伪造 IP 包
```

```
```
```

最核心的是 tcp\_syncookies=1：服务端收到 SYN 后不存半连接状态，而是把状态编码进 SYN+ACK 的 ack 字段里，客户端回 ACK 时服务端解密验证——零内存开销。

☒ 面试标准作答：seq 序号：TCP

把连接视为字节流，每个字节有绝对位置编号。握手时双方交换初始序号（ISN，Initial Sequence Number），建立后 seq 用来：① 接收方判断重复包、重排序、确认已收字节数（ACK = 期望收到的下一个 seq）；② 双方必须同步 ISN，否则无法确认边界。

SYN Flood 攻击：攻击者向目标发送大量伪造源 IP 的 SYN 包，目标为每个 SYN

分配半连接队列（backlog）项，超时才释放（默认 63 秒）。队列满后合法 SYN 被丢弃 → 拒绝服务。

防御手段：

① tcp\_syncookies=1（最关键）：服务端不保存半连接状态，而是将哈希(ISN, IP, port) 编入 SYN+ACK 的 ISN，客户端 ACK 回来时校验，无需内存；

② 加大 tcp\_max\_syn\_backlog（默认 128）；

③ 减少 tcp\_syn\_retries（默认 5 次，63s → 改 2 次约 9s）；

④ rp\_filter=1（严格模式）：Linux 验证源 IP 的反向路径，丢伪造包；

⑤ 上游 DDoS 清洗（云厂商/ISP）。

☒☒ 速记口诀：seq=字节编号+初始对齐；SYN 洪水=占半连接队列；防=syncookies+加大队列+rp\_filter。

## ☒ [5-深度-2] 五.1 VLAN

上次只讲了 VLAN 是什么，这次讲 VLAN 间路由——企业网络必设点。

☒ VLAN 之间怎么互通？单臂路由（Router-on-a-Stick）和三层交换机 SVI 各自什么场景用？

☒ 人话解释（看完必懂）：想象一个写字楼（交换机）分了 3 个公司（3 个 VLAN），各公司之间怎么互相串门？

方案 1：单臂路由——在路由器上开一根网线接到交换机，这根网线同时承载 3 个 VLAN 的流量（Trunk 口），

路由器在这根物理接口上划 3 个逻辑子接口（fa0/0.1 → VLAN10 网关，fa0/0.2 → VLAN20 网关...）。

问题：所有 VLAN 的流量都挤在这一根线上，容易堵；而且路由器转发性能不如交换机。

适合：小公司、VLAN 少（≤5 个）、临时应急。

方案 2：三层交换机 SVI——三层交换机本身就能当路由器（内置三层路由引擎），

不用额外接路由器，直接在交换机上给每个 VLAN 配一个虚接口（SVI，Switched Virtual Interface），

比如 interface Vlan-interface 10 → ip address 192.168.10.1 24 就把这个接口变成 VLAN10 的网关了。

优势：转发快（硬件 ASIC）、不占额外端口、同时支持二层交换和三层路由。

适合：中大型网络，VLAN 多，是现在主流方案。

H3C 配 SVI 示例：

```

system-view

vlan 100

interface Vlan-interface 100

ip address 192.168.100.1 255.255.255.0 → VLAN100 网关

ip route-static 0.0.0.0 0.0.0.0 10.0.0.1 → 默认路由指向出口

```

☒ 面试标准作答：VLAN 间路由的三种方式：

① 外部路由器 + 单臂路由：路由器子接口 + 交换机

Trunk，优点是便宜通用，缺点是单链路瓶颈，VLAN 多时转发慢；

② 三层交换机 SVI：Vlan-interface 虚接口作为 VLAN 网关，硬件转发无瓶颈，企业主流方案；

③ 防火墙 ACL + 路由：在 FW 上配每个 VLAN

的子接口做网关，同时做安全策略（推荐安全要求高的场景）。

H3C SVI 完整配置（上面代码基础上补充）：

```

```
interface Vlan-interface 100
ip address 192.168.100.1 255.255.255.0
dhcp select global → 如果要做 DHCP 网关
interface Vlan-interface 200
ip address 192.168.200.1 255.255.255.0
ip route-static 0.0.0.0 0 10.0.0.1 → 默认出口
```
```

注意：三层交换机要先 `undo stp enable`（可选项）确保各 VLAN 互通；Trunk 口必须 allow 所有 VLAN。

☒☒ 速记口诀：单臂路由=一根线干多个 VLAN，性能差；SVI=交换机当路由器，硬件转发，主流。Vlan-interface 就是网关接口。

#### ☒ [5-深度-3] 五.1 STP

上次只讲了 STP 根桥选举，这次讲 STP 安全特性——企业网必备配置。

☒ STP 的 PortFast、BPDU Guard、Root Guard、UplinkFast 是干嘛的？生产交换机上应该怎么配？

☒ 人话解释（看完必懂）：这几个特性都是 STP 出来后发现生产环境踩的坑，补的安全补丁：

- PortFast：接终端 PC/服务器的 Access 口，跳过 Listening+Learning 阶段直接进入 Forwarding。

为啥要？因为电脑开机要等网络（获取 DHCP），STP 默认要 50 秒才能通，PortFast 让它 1 秒就通。

⚠☒ 但接交换机/集线器上不能开——不然会形成临时环路。

- BPDU Guard：如果某个开了 PortFast 的端口突然收到 BPDU（有人插了个交换机上来），直接把这个端口封掉（err-disabled），防止环路入侵。

- Root Guard：在非根桥的指定端口上开，防止有个 priority 更小的恶意交换机插上来抢根桥。一旦收到更优的 BPDU，端口进入 root-inconsistent 状态（Blocking）。

- UplinkFast（RSTP 里叫 Backup Port）：Access 交换机上如果有两条上行到核心交换机，平时只走一条，另一条阻塞；主链路断了，备链路 1 秒内切上来（STP 默认要 50 秒）。

生产 H3C 配置示例：

```

```
interface GigabitEthernet 1/0/5 → 接 PC 的 Access 口
port access vlan 10
stp edged-port enable → PortFast（H3C 叫 edged-port）
stp bpdu-protection → BPDU Guard
```

interface GigabitEthernet 1/0/24 → 上行 Trunk 口
stp root-protection → Root Guard
、、、

☒ 面试标准作答：四大安全特性：

| 特性 | 作用 | 配置位置 |

|-----|-----|-----|

PortFast (edged-port)	跳过 Listening/Learning，直接 Forwarding	Access 口（接终端）
BPDU Guard (bpdu-protection)	收到 BPDU 立即 err-disabled	Access 口（配合 PortFast）
Root Guard (root-protection)	禁止更优 BPDU 进入，保护根桥	上行 Trunk/指定端口
UplinkFast (RSTP 内置 Backup)	上行链路故障时 1s 内切换	Access 交换机上行口

生产最佳实践：

- ① 所有接终端的 Access 口：PortFast + BPDU Guard；
- ② 所有上行 Trunk 口：Root Guard；
- ③ 核心交换机：设固定根桥（stp priority 0），不要让自动选举；
- ④ 用 RSTP 或 MSTP（别用 STP，收敛太慢）；
- ⑤ 如果端口被 err-disabled 了，手动恢复：interface → shutdown → undo shutdown；
- ⑥ 可以配 err-disabled auto-recovery 自动恢复。

☒☒ 速记口诀：PortFast=接终端的口秒通；BPDU Guard=接终端的口怕有人插交换机；Root Guard=保护根桥不被抢；UplinkFast=上行故障秒切。

☒ [5-深度-4] 五.1 OSPF

上次只讲了 OSPF 是链路状态协议，这次深挖 OSPF 的核心概念——面试官最爱考。

☒ OSPF 的 DR/BDR 是啥？为什么要有？DR/BDR 怎么选举？Stub Area 和 NSSA 有啥区别？

☒ 人话解释（看完必懂）：• DR/BDR = 广播网络里的「片区长」：

一个广播网段（比如一个 VLAN，里面有 5 台 OSPF 路由器）如果让每台都和另外 4 台交换 LSA，就要 $5 \times 4 / 2 = 10$ 条邻接关系；但选 1 个 DR（主片区长）+ 1 个 BDR（副片区长），其它路由器只和 DR/BDR 建立邻接，DR 把自己的 LSDB 发给所有人——邻接数降到 $2 \times 4 = 8$ 条，规模越大越省。DR/BDR 只在广播/P2MP 网络选举，P2P 不选。

• 选举规则：路由器启动时先比 OSPF 接口优先级（0-255，默认 1），大的当 DR；优先级 0 = 不参与选举；优先级相同再比 Router ID（启动时选最高 IP），大的胜；DR 挂了 BDR 自动顶上。注意：选举是一次性的（除非 DR/BDR 都挂了），新加的路由器不会抢。

• Stub Area vs NSSA：

两者都不让 Type-5 LSA（外部路由，比如从 BGP 或静态引入的）进来，减小 LSDB；

Stub Area 连 Type-4 都不让，A 路由器自动注入一条默认路由出去；

NSSA（Not So Stubby Area）更灵活——允许本 Area 的 ASBR 产生 Type-7 LSA，到 ABR

时自动转成 Type-5 发去骨干。适合「我自己有外部路由要发，但不想收到外面的」的场景。
记住：Stub = 完全不让进外部路由；NSSA = 只让本 Area 的外部路由出去，不让外面的进来。

☒ 面试标准作答：DR/BDR 机制：

- 目的：减少广播网段内的 OSPF 邻接数量（从 $n(n-1)/2$ 降到 $2n-2$ ）；
- 选举：接口 Priority（0=不参与）→ Router ID 最大者胜（不是最小！和 STP 根桥选举方向相反）；
- 特性：非抢占（现有 DR/BDR 挂了才重新选）；P2P 和 Loopback 不选举；

Area 类型：

| 类型 | 允许的 LSA | 适用场景 |

|-----|-----|-----|

| Normal Area | 所有 Type 1-5 | 一般业务区域 |

| Stub Area | Type 1-4，阻 5 | 末端区域，无 ASBR |

| Totally Stub | Type 1-3，阻 4+5（只一条默认） | 极端资源受限 |

| NSSA | Type 1-4 + Type 7（本 Area 引入） | 有本域 ASBR 又想阻外部 |

配置命令：area 1 stub / area 1 nssa。同一个 Area

里所有路由器配置必须一致，不一致邻接起不来。

☒☒ 速记口诀：DR/BDR=广播网片区长，Priority→RID

大选，非抢占。Stub=不让外部路由进来；NSSA=我能发出去但不让外面的进来。

☒ [5-深度-5] 五.1 + 五.2 H3C + NAT

上次只讲了 OSPF 理论，这次动手配 NAT + OSPF 联动——企业网必配。

☒ H3C 路由器上配 NAT：让内网 192.168.0.0/24 通过外网接口 Dialer0 上网，同时内网跑 OSPF 把自己网段发布出去。写出完整命令。

☒ 人话解释（看完必懂）：完整配置（H3C 路由器）：

```

system-view

interface GigabitEthernet 0/0 → 内网口

ip address 192.168.0.1 24

nat outbound 2000 → 绑 ACL 2000 做 PAT（端口地址转换）

interface Dialer0 → 外网口（PPPoE/4G/专线，假设已获取 IP）

ip address ppp-negotiate → 动态获取

nat outbound 2000

nat server protocol tcp global 0.0.0.0 80 inside 192.168.0.100 80

# 如果还要做服务器映射（外网 → 内网 Web 服务器）

acl basic 2000

rule 0 permit source 192.168.0.0 0.0.0.255 → 只允许这个网段上网

rule 5 deny

ospf 1 router-id 10.0.0.1 → OSPF 进程

area 0

network 10.0.0.1 0.0.0.0 → 骨干区（假设 OSPF 邻居都在骨干区）

network 192.168.0.1 0.0.0.0 → 把内网网段发布到 OSPF

default-route-advertise always → 向外发布默认路由（可选）

ip route-static 0.0.0.0 0.0.0.0 Dialer0 → 默认路由指外网口

、、、

关键：nat outbound 2000 是 PAT（Port Address Translation，多对一），路由器用自己 Dialer0 的公网 IP + 不同端口号映射内网多台机器。OSPF network 192.168.0.1 0.0.0.0 让其它 OSPF 路由器学到这个内网网段。

☑ 面试标准作答：完整 H3C NAT + OSPF 配置如上。

nat outbound 两种形式：

- nat outbound acl-number：PAT，用出接口 IP 做转换（最常见，多对一）；
- nat outbound acl-number address-group x：多公网 IP 轮转（需要公网 IP 池）。

nat server（服务器映射，静态）：nat server protocol tcp global 公网IP 80 inside 内网IP 80，把外网 80 端口固定映射到内网 Web 服务器。

OSPF 配合 NAT 的坑：

- ① OSPF 的邻居协商（Hello 包）是组播 224.0.0.5/6，\*\*不会被 NAT 转换\*\*，所以 OSPF 邻居必须在 NAT 内网侧；
- ② 如果 OSPF 邻居中有一侧在 NAT 外部（比如对端是公网），要用 Tunnel（GRE 隧道）包起来再跑 OSPF；
- ③ default-route-advertise always 强制发布默认路由，便于其它 OSPF 路由器走 NAT 出口上网。

☑☑ 速记口诀：NAT=nat outbound 2000（PAT上网）/nat server（静态映射）；OSPF 发布网段用 network + 反掩码。NAT不转组播，OSPF 邻居要在内网侧。

## 六、综合场景（3 道）

☒☒ 本方向共 3 道深度题，全部是上一版没出的点。

### ☒ [6-深度-1] 综合

MGR 自动切换是高频故障，上次只讲了 SECONDARY 掉线，这次讲 PRIMARY 切换后的坑。

☒ MGR 集群的 PRIMARY 自动切换到另一个节点之后，业务突然访问不了了——报「表不存在」或者「权限错误」，怎么排查？

☒ 人话解释（看完必懂）：MGR 自动切换后业务挂了，常见 5 大原因按概率排序：

① 连接没重连到新 PRIMARY：你的 JDBC 连接池可能还连在旧

PRIMARY（已变成只读），一写就报「只读不允许写」。如果用了 ProxySQL，检查 mysql\_replication\_hostgroups 有没有正确识别新 PRIMARY；没 ProxySQL 就只能在应用层改连接字符串——直接说「这就是生产必须用 ProxySQL 的原因」。

② 新 PRIMARY 上缺表/缺权限：说明旧 PRIMARY 上做过 DDL（CREATE/ALTER

TABLE）但没有同步成功。MGR 的 DDL 是严格一致的——只要节点活着就应该同步，如果新 PRIMARY 缺表，说明这个 DDL 执行时有 errant transaction。排查：在新 PRIMARY 上 `SHOW CREATE TABLE xxx;` → 没有就从旧 PRIMARY（如果还活着）导过来。

③ GRANT 没同步：MySQL 8.0 的用户信息在 mysql 系统表里，MGR 模式下 GRANT 应该同步；但 mysql 系统表的 DDL 可能被当成 errant。排查：新 PRIMARY `SELECT user,host FROM mysql.user;` 对比旧 PRIMARY。

④ 应用指向了错误的 hostgroup：ProxySQL 的 mysql\_query\_rules 还把写请求路由到旧 hostgroup。排查 ProxySQL admin 接口 `SELECT \* FROM mysql\_servers;` 看 hostgroup 归属是否正确。

⑤ 新 PRIMARY 之前是只读状态没切过来：MGR 自动切换失败（比如 split-brain），可以在所有节点执行 `SHOW VARIABLES LIKE 'group\_replication\_role';` 看谁是 PRIMARY。

☒ 面试标准作答：PRIMARY 自动切换后业务不可用的排查清单：

Step 1：先确认集群真实状态（在每个节点都执行）：

```
```sql
```

```
SELECT * FROM performance_schema.replication_group_members;
```

```
SHOW VARIABLES LIKE 'group_replication_role'; -- PRIMARY / SECONDARY
```

```
SHOW VARIABLES LIKE 'read_only'; -- PRIMARY 应该是 OFF
```

```
```
```

Step 2：确认应用连的是谁：

```
mysql -e "SHOW PROCESSLIST;" | grep -v system
```

看 Connection 来源 IP 的 Host 是旧 PRIMARY 还是新 PRIMARY。如果是旧的就是连接池没刷新。

Step 3: 用 PRIMARY 上 `mysqlbinlog --base64-output=DECODE-ROWS -v` 检查 DDL，看新 PRIMARY 缺的表有没有对应的 CREATE TABLE 语句——没有说明是 errant transaction，需要在新 PRIMARY 上手动补建（从旧 PRIMARY mysqldump 过来）。

Step 4: ProxySQL 侧确认：

```
```sql
```

```
SELECT hostgroup_id,hostname,port,status FROM runtime_mysql_servers;
```

```
SELECT rule_id,match_digest,destination_hostgroup FROM runtime_mysql_query_rules;
```

```
```
```

根因总结：90% 是「连接池指向旧 PRIMARY」，解决办法：① JDBC 加 connection validation query + failover；② 用 ProxySQL 自动路由（强烈推荐）。MGR 自动切换时应用层必须有自动重连机制。

☒☒ 速记口诀：PRIMARY 切换后先查三点：集群成员状态→应用连的谁→ProxySQL hostgroup 归属。90% 是连接池没刷新，靠 ProxySQL 自动路由解决。

#### ☒ [6-深度-2] 综合

技术支持经常遇到客户强推全量——面试考你的沟通技巧和风险意识。

☒ 客户（甲方老板）坚持要直接全量发布新版本，不做灰度，还说「上一个版本灰度了也没发现问题」。你作为技术支持怎么沟通？

☒ 人话解释（看完必懂）：分三步：

① 先共情，不说教：「理解您想尽快上线的心情，我也希望这次一发就中，那我们来分析一下如果万一有问题的话代价是什么？」

② 用具体数字而不是空喊风险：「我们这个版本是 DB schema 变更（加了索引） +

新的异步接口，如果全量发挂了：- 数据库锁表影响 2000 用户写操作；

- 回滚需要把老版本镜像拉回来 + DB 回滚脚本重跑，整个过程预计 30-45 分钟；

- 上次灰度 10% 流量就发现了 X 模块在老数据上的边界问题，这次我们加了修复脚本，但 Y 模块是新加的，还没在大流量上测过。」

③ 给台阶 + 小妥协：「要不我们这样——比上次灰度规模大一点，直接 50% 灰度 1 小时，观察 CPU/DB QPS/错误率，如果一切正常再切剩下

50%。这样即使挂了影响范围也在可控，而且您整体上线速度只慢 1 小时，比全量发然后挂 30 分钟强。」

关键点：别跟客户说「不行」，要说「全量发的话风险是 A，灰度发的话风险降为 B 且时间只多 X 分钟」，让客户自己权衡。

☒ 面试标准作答：沟通框架：共情 → 风险量化 → 给出替代方案（比客户预期好一点）：

错误示范：「不行，必须灰度，全量有风险！」（客户反感，觉得你拖他节奏）

正确示范：

- ① 共情开场：「我懂您想尽快上线，这也是我们共同目标。上次灰度 10% 发现 X 问题，这次确实修复了，但我分析了下这次变更是 DB schema 升级 + 新增异步任务，有几个点我们没在生产全流量上跑过」；
- ② 风险量化：「全量发挂了 → 影响范围 100% 用户 → 回滚 30 分钟 → 客服电话打爆；灰度 50% 挂了 → 影响 50% → 回滚 15 分钟 → 可以容忍」；
- ③ 替代方案：「要不这次我们灰度比例调高到 50%（让一步），观察 30 分钟，如果 CPU/错误率都 OK 就全量切。整个上线只多 30 分钟，但把风险从『大事故』降到『可控』」；
- ④ 留客户决策：「当然最终您说了算，我这边只是把风险摆出来」（不越权）。
- 底线：如果是高危变更（DB DROP/ALTER ALL/加密逻辑变更），态度要硬——「这次是 DB 表结构变更，万一有 bug 全量恢复成本极高，必须灰度。这是我们团队 3 年没出过 P0 事故的底线」。

☒☒ 速记口诀：沟通三要素：共情+风险量化+给好台阶。不说「不行」，说「这样更好」。

#### ☒ [6-深度-3] 综合

TIME\_WAIT 是运维高频问题，和上次的 CPU/内存/磁盘并称「运维四大金刚」。

☒ 服务器上发现 netstat 有 30000 个 TIME\_WAIT 连接，占满端口、业务连不上数据库了。怎么处理？

☒ 人话解释（看完必懂）：先理解为什么会有 TIME\_WAIT：

TIME\_WAIT 是主动关闭连接的一方（客户端/服务端谁先 close 谁主动）在挥手完成后要等 2 分钟（2MSL），让残留报文消散。如果你的服务端处理了大量短连接（比如客户端请求一个页面就关闭），每个连接都占一个端口 2 分钟，30000 个就耗尽了 ephemeral port range（默认 28000-60999，约 3 万）。

处理分两步：

① 紧急止血（临时）：

```
```bash
```

```
sysctl -w net.ipv4.tcp_tw_reuse=1 # 紧急：允许 TIME_WAIT 连接被新请求复用
```

```
sysctl -w net.ipv4.ip_local_port_range='1024 65535' # 扩大端口范围
```

```
```
```

② 根治方案（长期）：

- 应用层：用连接池！别每次请求都建新连接（Java HikariCP、Python requests.Session、MySQL pool）；

- 服务端（如果是服务端主动关闭导致的 TIME\_WAIT）：让客户端主动 close（反过来就变成 FIN\_WAIT2），但客户端没这能力……所以还是连接池解决；

- 内核调优（加到 /etc/sysctl.conf）：

```
net.ipv4.tcp_tw_reuse=1（复用）
```

net.ipv4.tcp\_fin\_timeout=30 (主动关闭方 FIN\_WAIT2 从 60s 降到 30s)  
net.ipv4.ip\_local\_port\_range='1024 65535' (扩大范围)  
net.ipv4.tcp\_max\_tw\_buckets=180000 (最大 TIME\_WAIT 数量, 默认 18 万)  
- 验证: sysctl -p 生效; ss -s 看 TIME\_WAIT 数量。

☒ 面试标准作答: 根因: TIME\_WAIT 数量 = 主动关闭连接的频率 × 2MSL (120s)。短连接风暴  
→ 端口耗尽。

紧急处理:

- ① tcp\_tw\_reuse=1 (最关键, 内核 ≥ 4.1 支持): 让新出站请求可以复用 TIME\_WAIT 状态的端口;
  - ② 扩大 ip\_local\_port\_range (默认 32768-60999 → 改 1024-65535);
  - ③ 如果是服务端产生大量 TIME\_WAIT → 让应用改为 HTTP keep-alive / 数据库连接池 (根治)。
- 永久内核调优 (/etc/sysctl.conf):

```\n

```
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_tw_recycle = 0 # 4.12 内核后已废弃, 不要开
net.ipv4.tcp_fin_timeout = 30
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_max_tw_buckets = 262144
net.core.somaxconn = 65535 # 监听队列也调大
```\n
```

不要用 tcp\_tw\_recycle (已废弃): 它会丢弃来自同一 NAT IP 的新连接 (因为端口号重用导致 seq 检查不通过), 4.12 内核后这个参数已经被移除了。正确的做法是 tcp\_tw\_reuse。

☒☒ 速记口诀: TIME\_WAIT 根因: 短连接风暴。紧急用 tcp\_tw\_reuse; 根治用连接池 + keep-alive。别碰 tcp\_tw\_recycle!



## 两版组合使用建议（明天就面试，没时间全背？看这里）

上一版（23 道基础+速记）：快速搭建知识框架，覆盖 JD 所有条目的第一层考点；

本版（20 道深度追问）：准备好被面试官「继续追问」时的答法；

组合背的优先级：

① 先背上一版 PDF 里的速记口诀表（最后一页，23 行）→ 唤醒记忆；

② 再背本版里标记 JD 核心关键词的题：

- MySQL redo/undo 两阶段提交（JD 要「内核原理」）
- MVCC Read View（面试官必追问 RC vs RR）
- ProxySQL 配置片段（JD 要「熟练使用」）
- K8s 三种探针（JD 要「熟悉技术栈」）
- Zabbix Trigger + Prometheus Alerting Rules（JD 要「精通至少两种」）
- H3C NAT+OSPF 完整命令（JD 要「独立配置 H3C」）
- MGR PRIMARY 切换后业务故障（高频生产场景）
- TIME\_WAIT 根治（运维日常）

临场加分项：主动说「我知道 MGR 用 Paxos，不过我现在也在看 Raft 协议在云原生里的应用」（体现学习热情）。